

AhamX Bodhi

Where I Becomes Infinite

Output Formatting Control

Module 2.3: Prompt Engineering | Prof. Sashikumar Ganesan |
IISc Bengaluru



Session Overview

What You Will Learn

- Why output format control matters in production
- Prompt-based formatting techniques
- Structured output modes (JSON mode, function calling)
- Output parsing strategies and libraries

Session Details

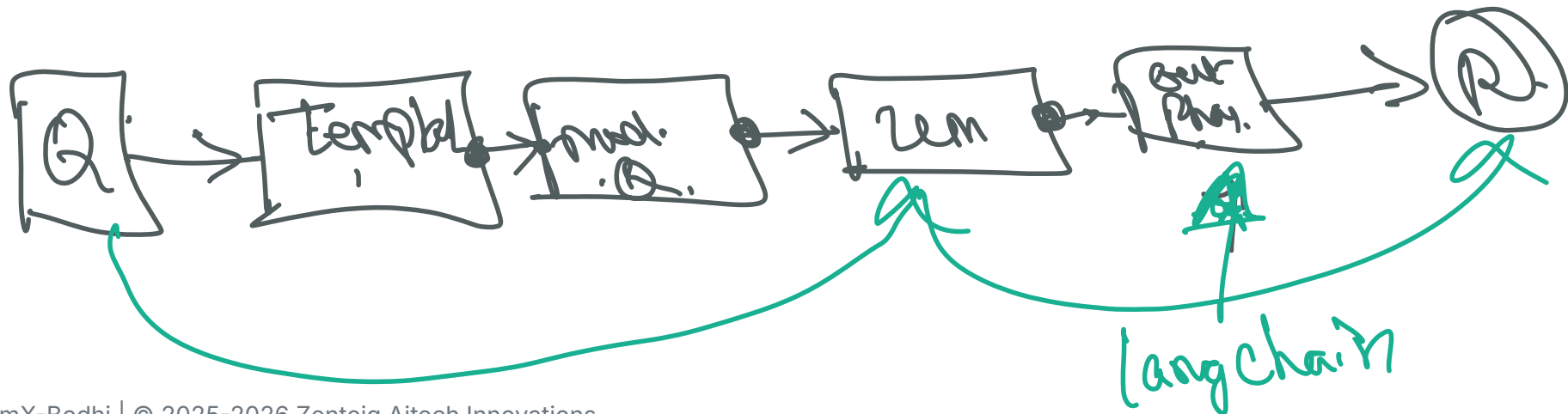
- **Duration:** 8 minutes
- **Bloom's Level:** B3 — Apply
- **Difficulty:** L2 — Intermediate
- **Module:** 2.3 Prompt Engineering
- **Prereq:** Prompt Engineering Principles



Learning Objectives

By the End of This Session You Will Be Able To

- **Explain** why uncontrolled output format breaks automated pipelines
- **Apply** prompt-based format instructions for JSON, Markdown, and plain text
- **Use** structured output APIs (JSON mode, function calling) for guaranteed structure
- **Implement** output parsing with Pydantic and LangChain output parsers







The Output Format Problem

Why Uncontrolled Output Breaks Pipelines

- LLMs default to verbose, narrative prose when no format is specified
- Downstream code expecting JSON will fail if the model adds conversational preamble
- Output length and structure vary between calls — not deterministic by default
- Parsing free-form text is fragile: field names, delimiters, and order may change

In production pipelines, output format is as important as output content. A correct answer in the wrong format is useless to automated downstream systems.



Prompt-Based Format Control

Specifying Format in the Prompt

- **JSON:** "Respond only with a valid JSON object. No additional text. Schema: {...}"
- **Markdown:** "Format your response in Markdown with H2 headers for each section."
- **Numbered list:** "Provide exactly 5 numbered steps. Each step in one sentence."
- **Table:** "Present the comparison as a Markdown table with columns: Feature | Option A | Option B."
- **Length:** "Respond in exactly two sentences." / "Maximum 150 words."



JSON Mode and Structured Output APIs

Beyond Prompt Instructions: API-Level Guarantees

- **JSON Mode (OpenAI):** Setting `response_format: {type: "json_object"}` guarantees valid JSON output. The model will never produce non-JSON text.
- **Structured Outputs (OpenAI GPT-4o):** Provide a JSON Schema; the model is constrained to match it exactly.
- **Function Calling:** Define a function schema; the model fills in arguments. Most reliable method for structured extraction tasks.

When to Use API-Level vs. Prompt-Level Format Control

- Use API-level for guaranteed structure in automated pipelines
- Use prompt-level for flexibility, Markdown output, or quick prototypes



Structured Outputs with Pydantic (OpenAI)

```

1  from openai import OpenAI
2  from pydantic import BaseModel
3
4  class ReviewAnalysis(BaseModel):
5      sentiment: str          # "Positive", "Negative", "Neutral"
6      key_issue: str          # Main topic of the review
7      urgency_score: int      # 1-5, where 5 is most urgent
8
9  client = OpenAI()
10 response = client.beta.chat.completions.parse(
11     model="gpt-4o-2024-08-06",
12     messages=[
13         {"role": "system", "content": "Analyze the customer review."},
14         {"role": "user", "content": "The product broke in 2 days!"}
15     ],
16     response_format=ReviewAnalysis,
17 )
18 result = response.choices[0].message.parsed
19 # result.sentiment = "Negative"
20 # result.urgency_score = 5

```



Output Parsers in LangChain

LangChain Output Parser Types

- **StrOutputParser:** Returns raw string; minimal processing
- **JsonOutputParser:** Parses JSON string to Python dict; includes format instructions
- **PydanticOutputParser:** Maps JSON to typed Pydantic model; validates schema
- **CommaSeparatedListOutputParser:** Splits comma-delimited output to Python list
- **StructuredOutputParser:** Uses ResponseSchema to define expected fields; injects format instructions automatically into the prompt



Output Format Control Methods: Comparison

Method	Guarantee	Flexibility	Best Use
Prompt instructions	Soft (may fail)	High	Prototyping, Markdown
JSON Mode (API)	Hard (valid JSON)	Medium	Automated pipelines
Structured Outputs	Hard (schema match)	Low	Production extraction
Function Calling	Hard (typed args)	Low	Tool use, APIs
LangChain Parsers	Soft + retry	Medium	LLM app development





Application: LLM-to-API Integration

Structured Output Enables System Integration

Enterprise automation platforms (Zapier AI, Make.com AI, UiPath AI) use structured LLM outputs to feed data directly into downstream APIs and databases. Guaranteeing JSON schema prevents integration failures in automated workflows.

Production Pattern

- LLM extracts data from unstructured email/document
- Structured output matches the target CRM API schema exactly
- No manual mapping or transformation required
- Pydantic validation catches any remaining schema mismatches before API call



Application: AI-Powered Dashboard Narratives

Structured Text for BI Tools

Business intelligence platforms (Tableau Pulse, Power BI Copilot, Looker AI) use controlled output formatting to generate narrative summaries alongside charts. Markdown format instructions ensure consistent heading structure across all reports.

Format Control for Reporting

- Fixed Markdown template ensures consistent H2/H3 section structure
- Length constraints guarantee summaries fit dashboard cards
- Structured JSON output populates chart annotations programmatically



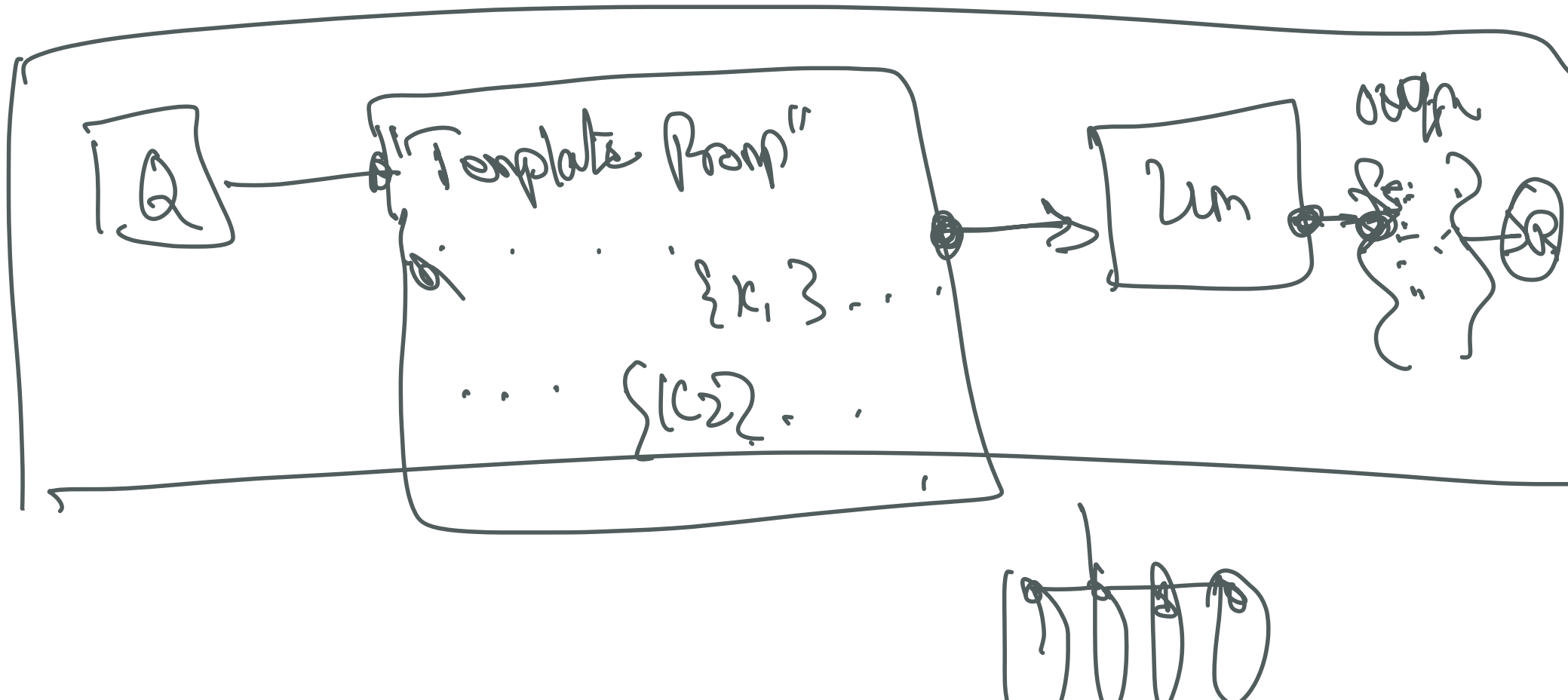
Application: Multi-Modal Data Pipelines

From Images and Documents to Structured Records

Document intelligence platforms (AWS Textract + Bedrock, Azure AI Document Intelligence, Google Document AI) extract structured data from scanned documents using LLMs with strict output schema enforcement.

Why Schema Enforcement is Critical

In healthcare, finance, and legal contexts, a misaligned output field can cause regulatory non-compliance, billing errors, or contract disputes. API-level schema guarantees are non-negotiable in these domains.





Key Takeaways

Core Concepts to Remember

- Uncontrolled LLM output format breaks automated pipelines; format control is essential
- Prompt-level instructions provide flexible but soft formatting guarantees
- JSON Mode, Structured Outputs, and Function Calling provide hard API-level guarantees
- Pydantic and LangChain output parsers add schema validation and retry logic
- In production, always choose the strongest format guarantee your use case supports

AhamX Bodhi

Where I Becomes Infinite